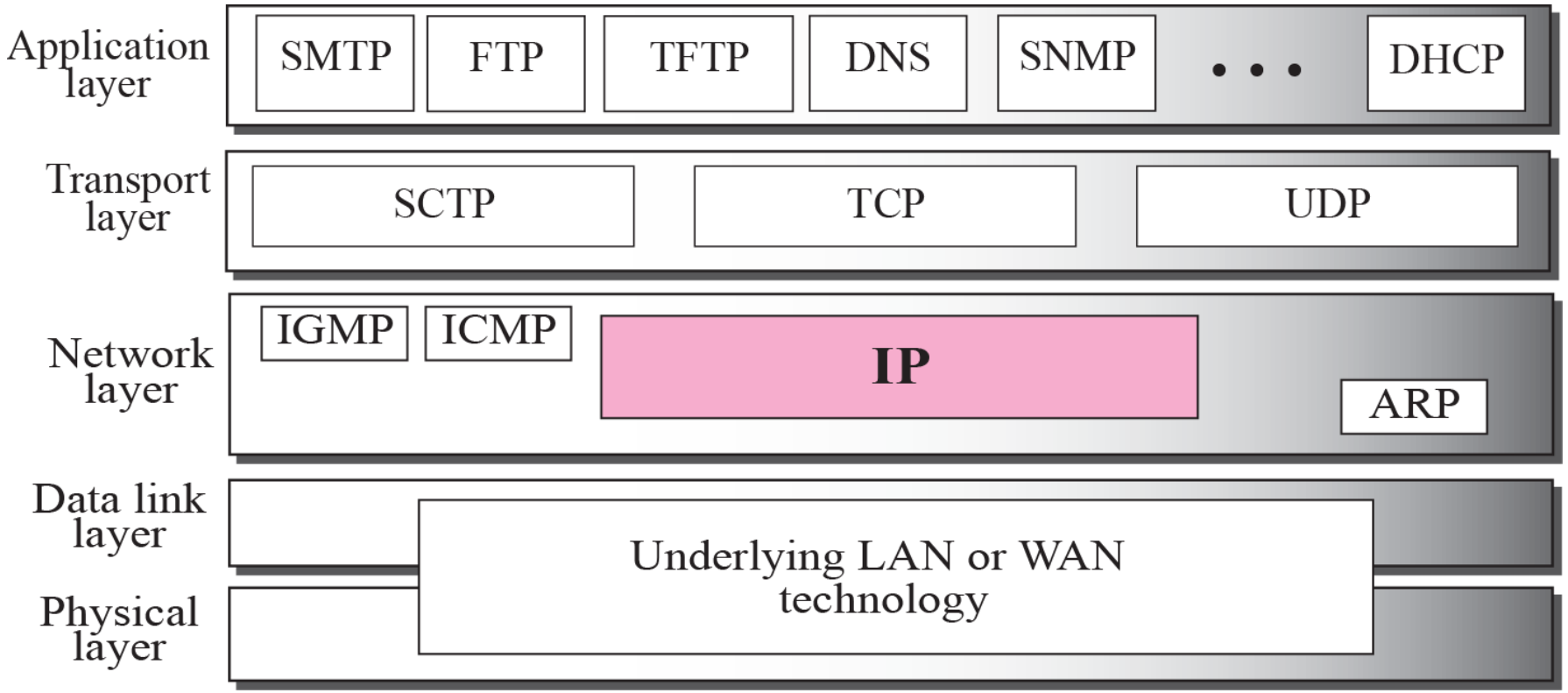
EETS 7302 TCP/IP Network Administration

Session 6
IPv4 & ARP

IP

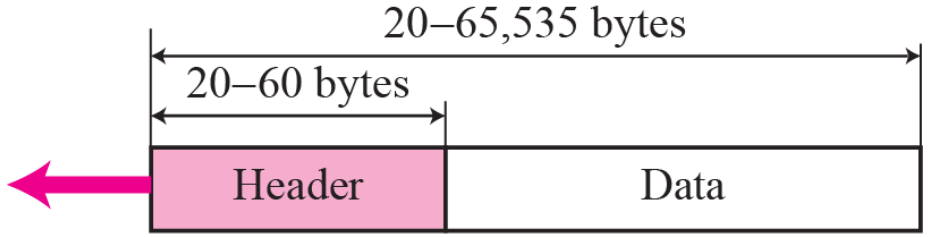
The Internet Protocol (IP) is the transmission mechanism used by the TCP/IP protocols at the network layer.

Position of IP in TCP/IP protocol suite

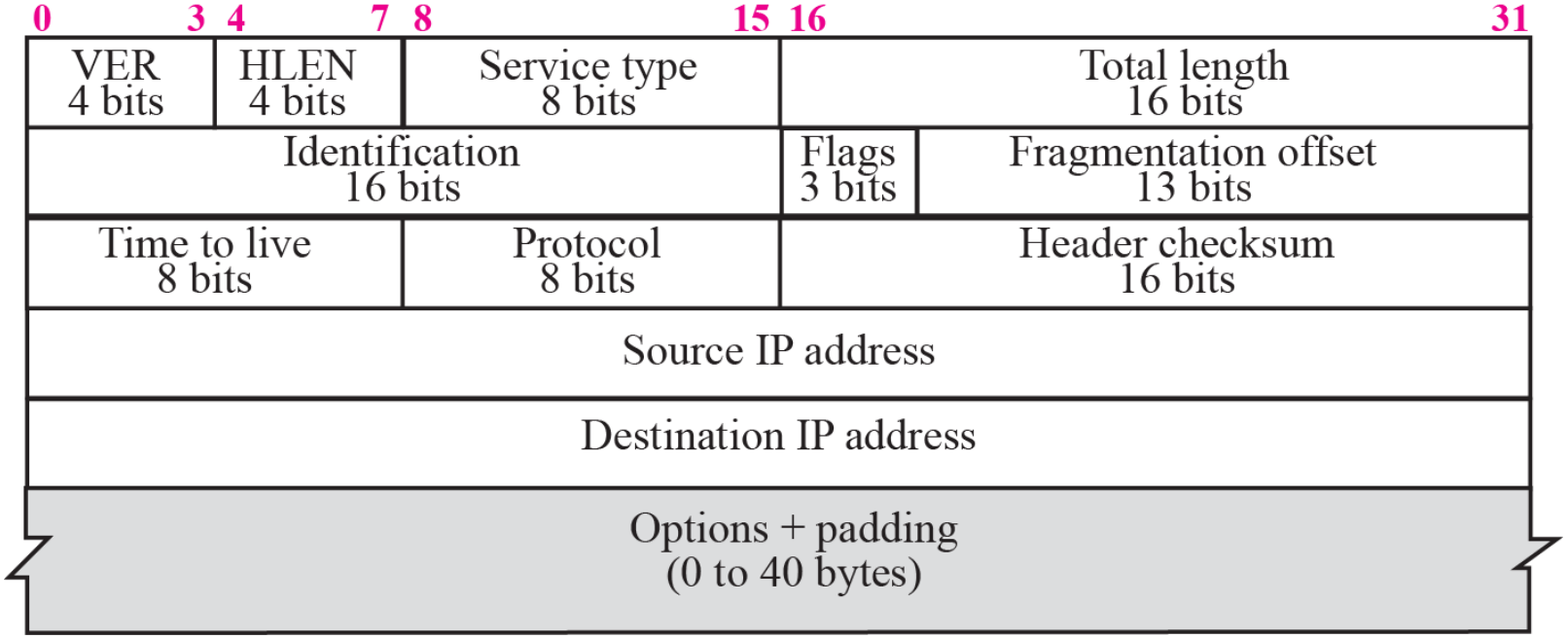


DATAGRAMS

- Packets in the network (internet) layer are called ***datagrams***.
- A datagram is a variable-length packet consisting of two parts:
  - Header: 20 to 60 bytes in length and contains information essential to routing and delivery.
  - Data



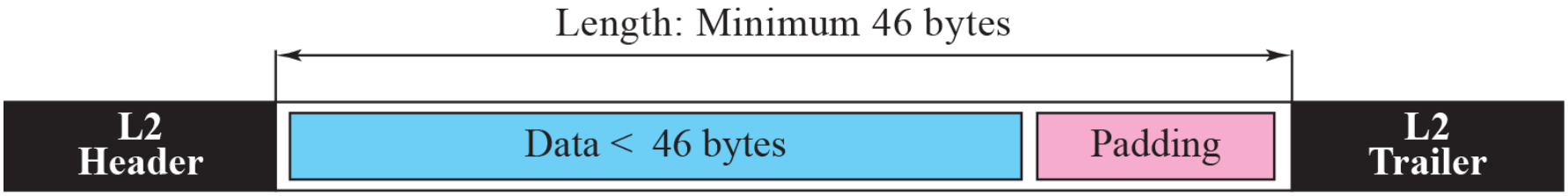
a. IP datagram



b. Header format

The total length field defines the total length of the datagram including the header.

Encapsulation of a small datagram in an Ethernet frame



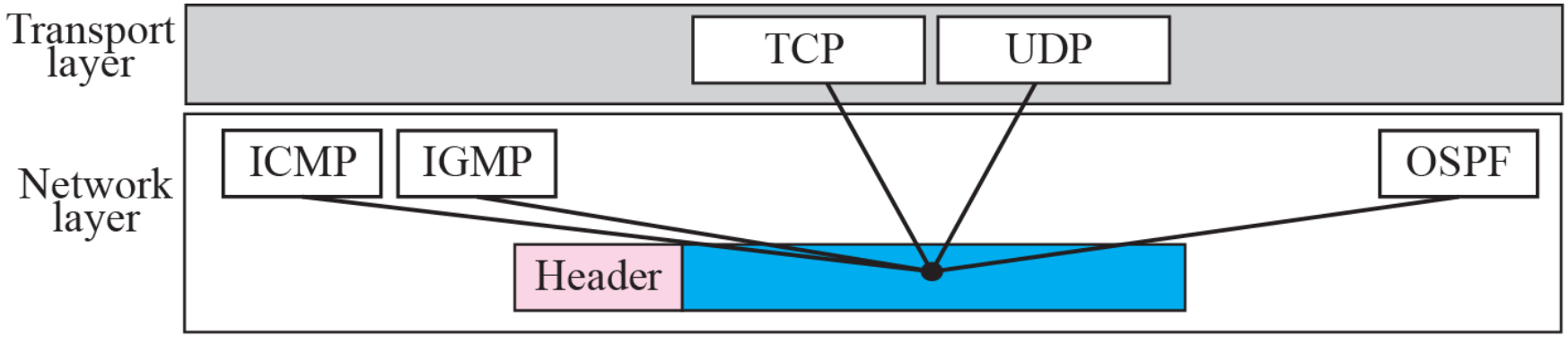

Transport layer
TCP
UDP
Network layer
ICMP
IGMP
OSPF
Header

Table 7.2 *Protocols*

Value	*Protocol*	*Value*	*Protocol*
1	ICMP	17	UDP
2	IGMP	89	OSPF
6	TCP		

Example 5 min exercise

An IP packet has arrived with the first few hexadecimal digits as shown below:

45000028000100000102 . . .

How many hops can this packet travel before being dropped? The data belong to what upper layer protocol?

Example 5 min exercise

An IP packet has arrived with the first few hexadecimal digits as shown below:

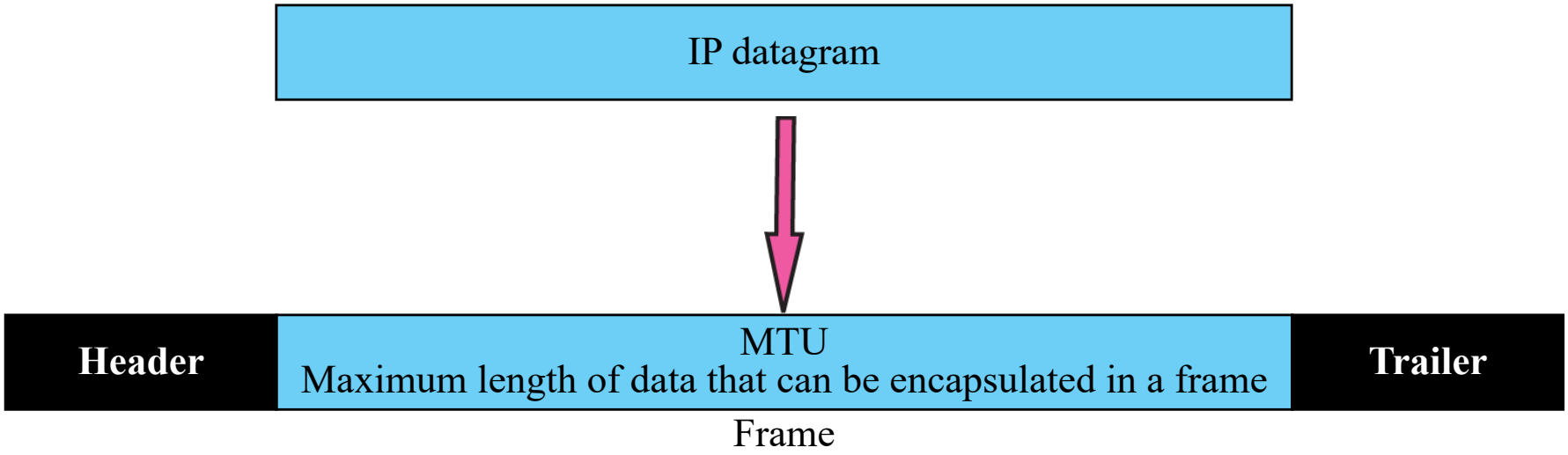
How many hops can this packet travel before being dropped? The data belong to what upper layer protocol?

Solution

To find the time-to-live field, we skip 8 bytes (16 hexadecimal digits). The time-to-live field is the ninth byte, which is 01. This means the packet can travel only one hop. The protocol field is the next byte (02), which means that the upper layer protocol is IGMP

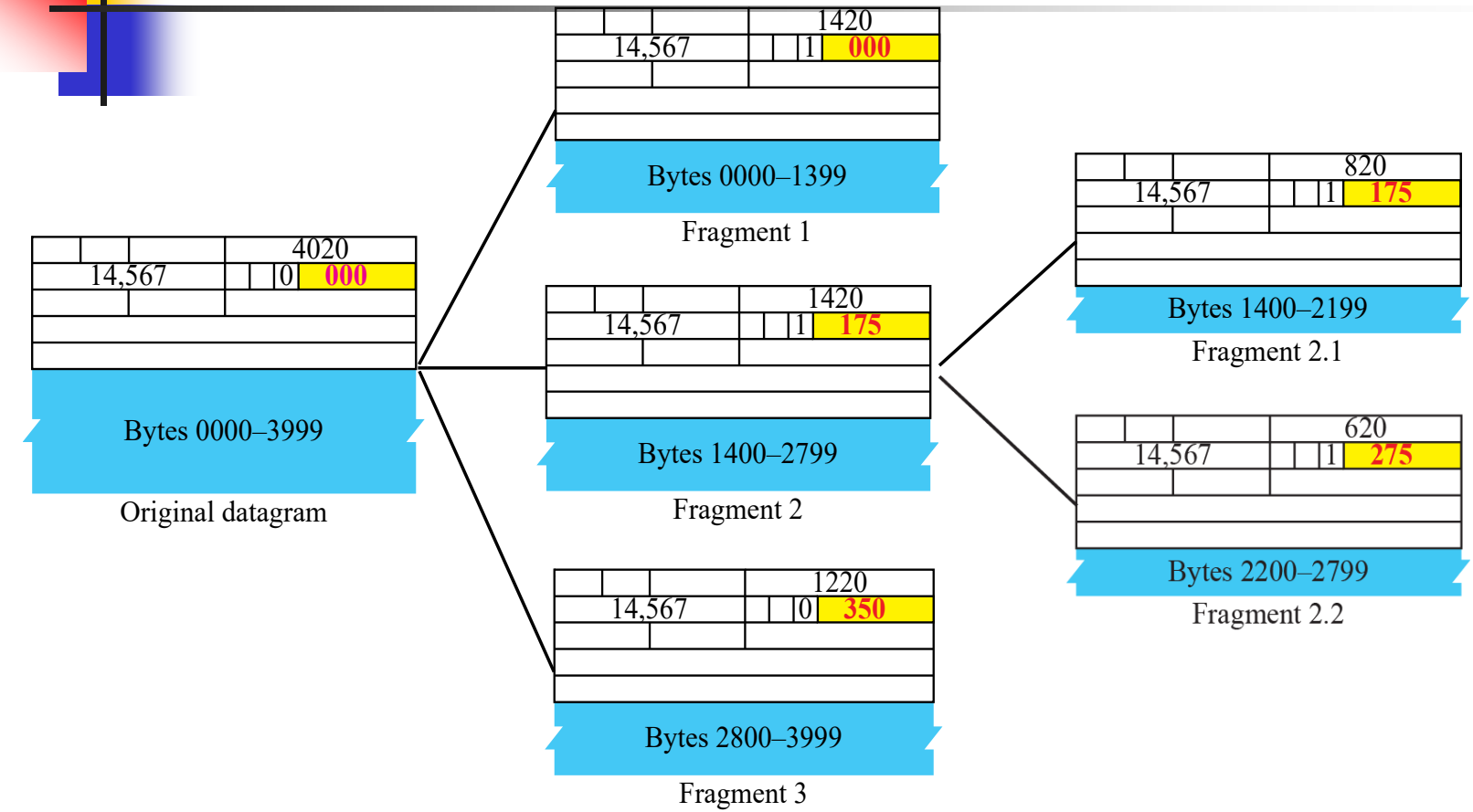
FRAGMENTATION

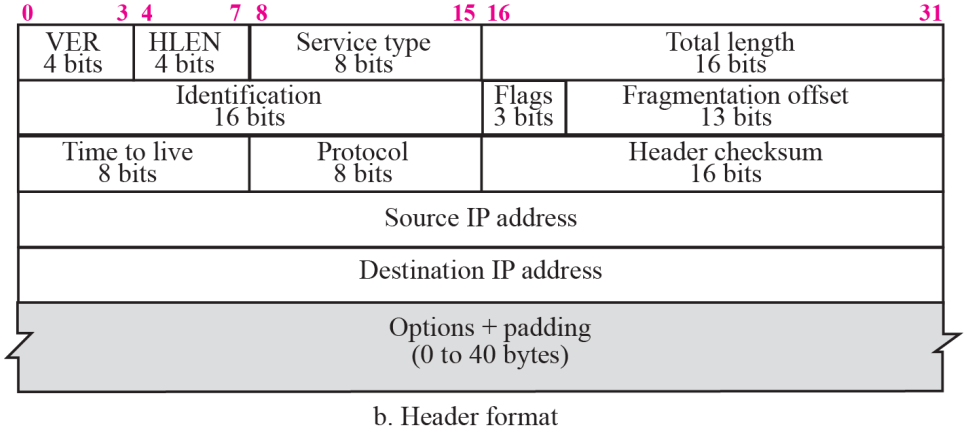
A datagram can travel through different networks. Each router decapsulates the IP datagram from the frame it receives, processes it, and then encapsulates it in another frame. The format and size of the received frame depend on the protocol used by the physical network through which the frame has just traveled. The format and size of the sent frame depend on the protocol used by the physical network through which the frame is going to travel.



Only data in a datagram is fragmented.

Detailed fragmentation example





b. Header format

Example 5 min exercise

A packet has arrived with an M bit value of 0. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

Example

A packet has arrived with an M bit value of 0. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

Solution

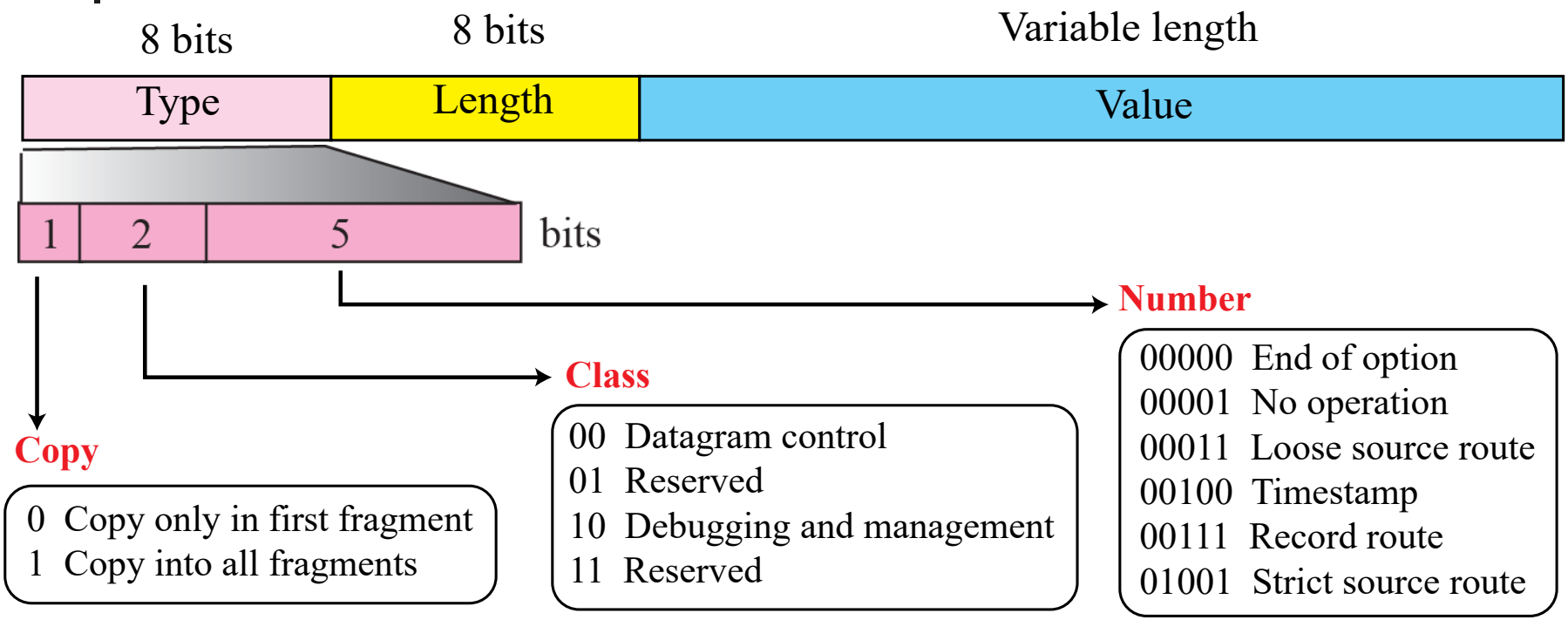
If the M bit is 0, it means that there are no more fragments; the fragment is the last one. However, we cannot say if the original packet was fragmented or not. A nonfragmented packet is considered the last fragment.

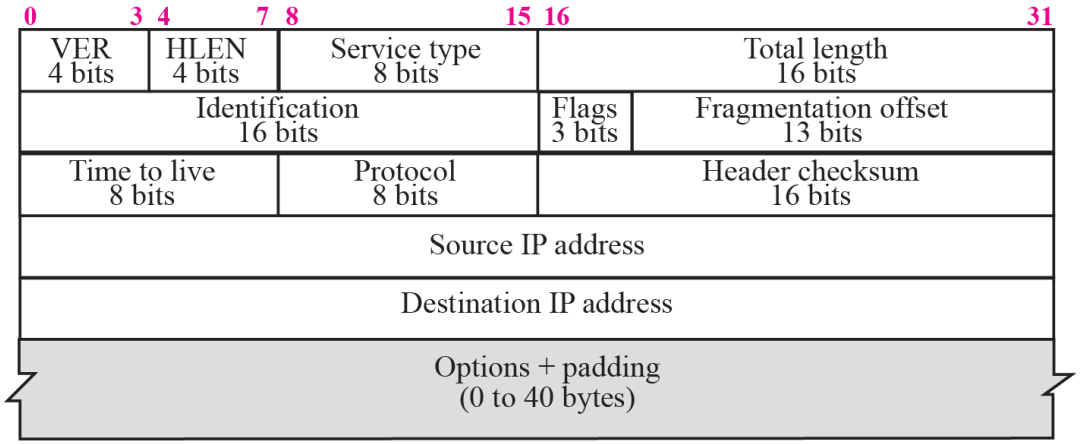
OPTIONS

The header of the IP datagram is made of two parts: a fixed part and a variable part. The fixed part is 20 bytes long and was discussed in the previous section. The variable part comprises the options, which can be a maximum of 40 bytes.

used for network testing and debugging.

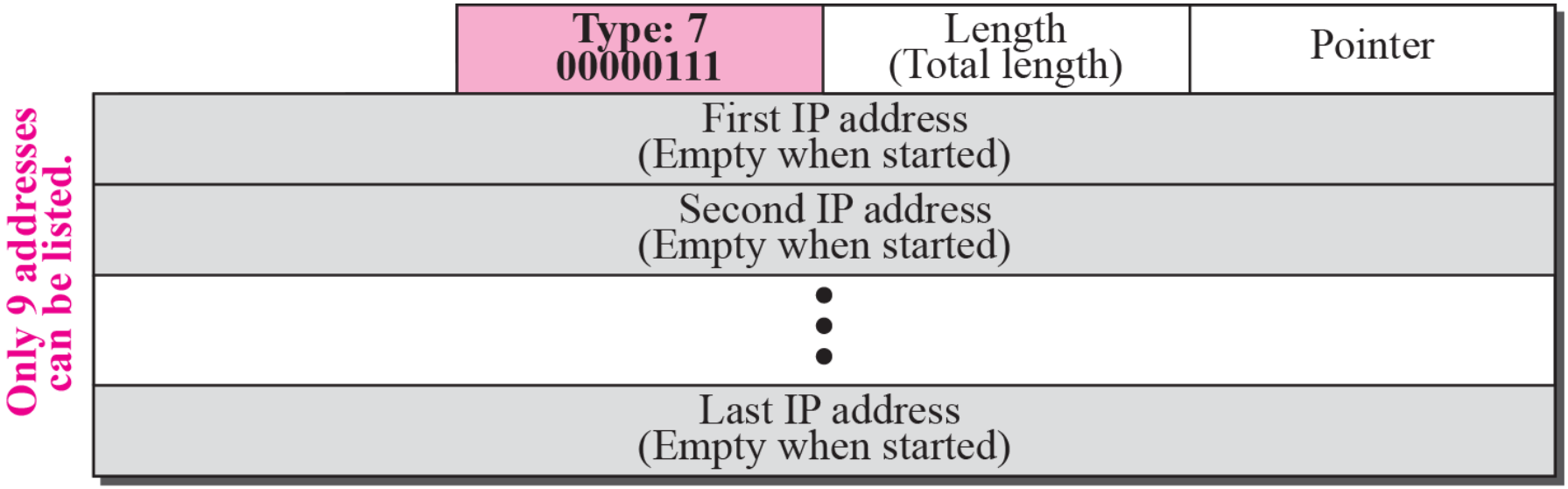
Option format



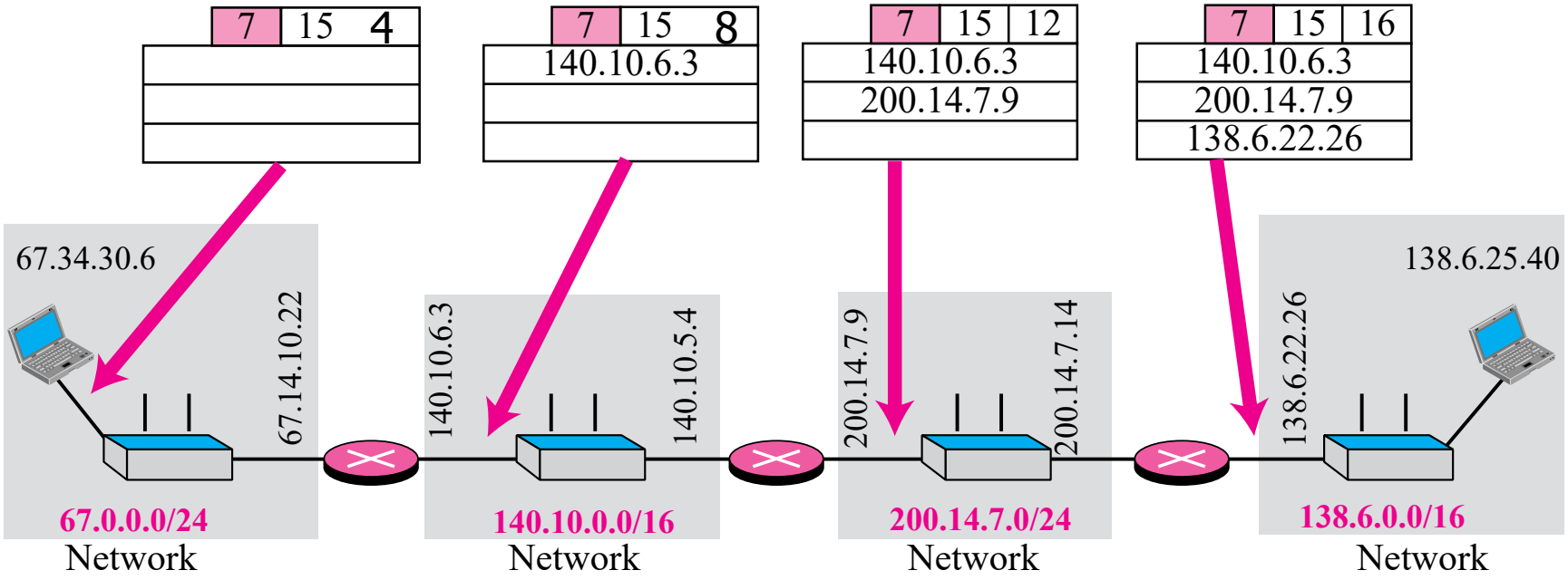


b. Header format

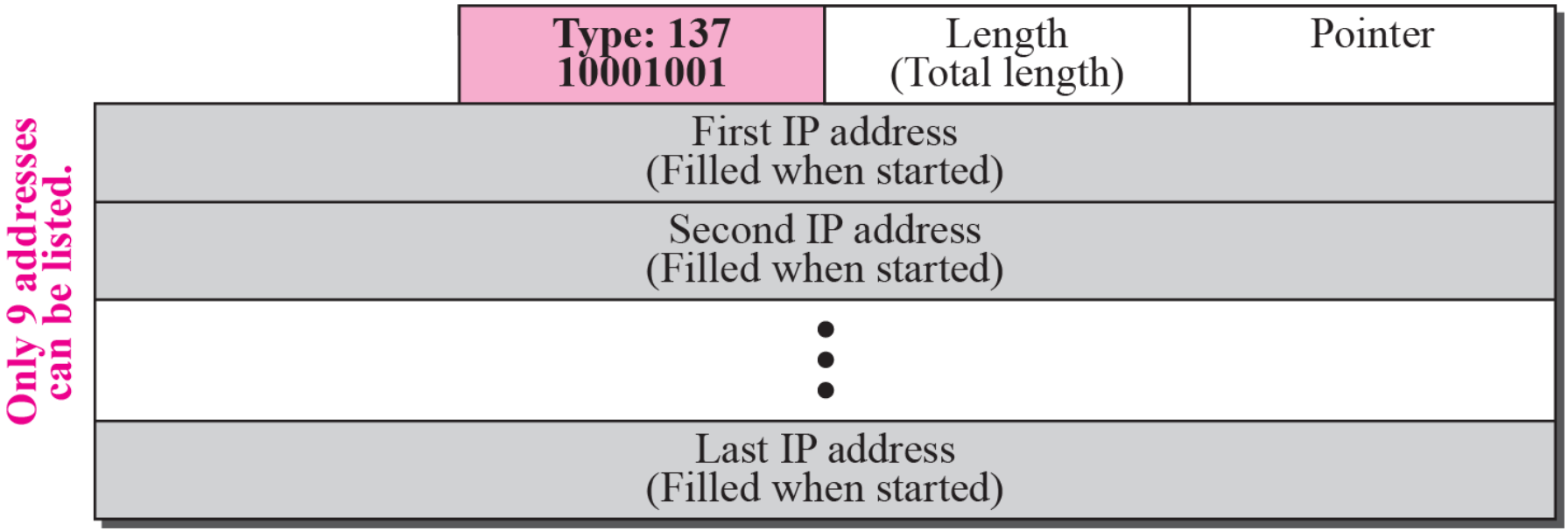
Record-route option



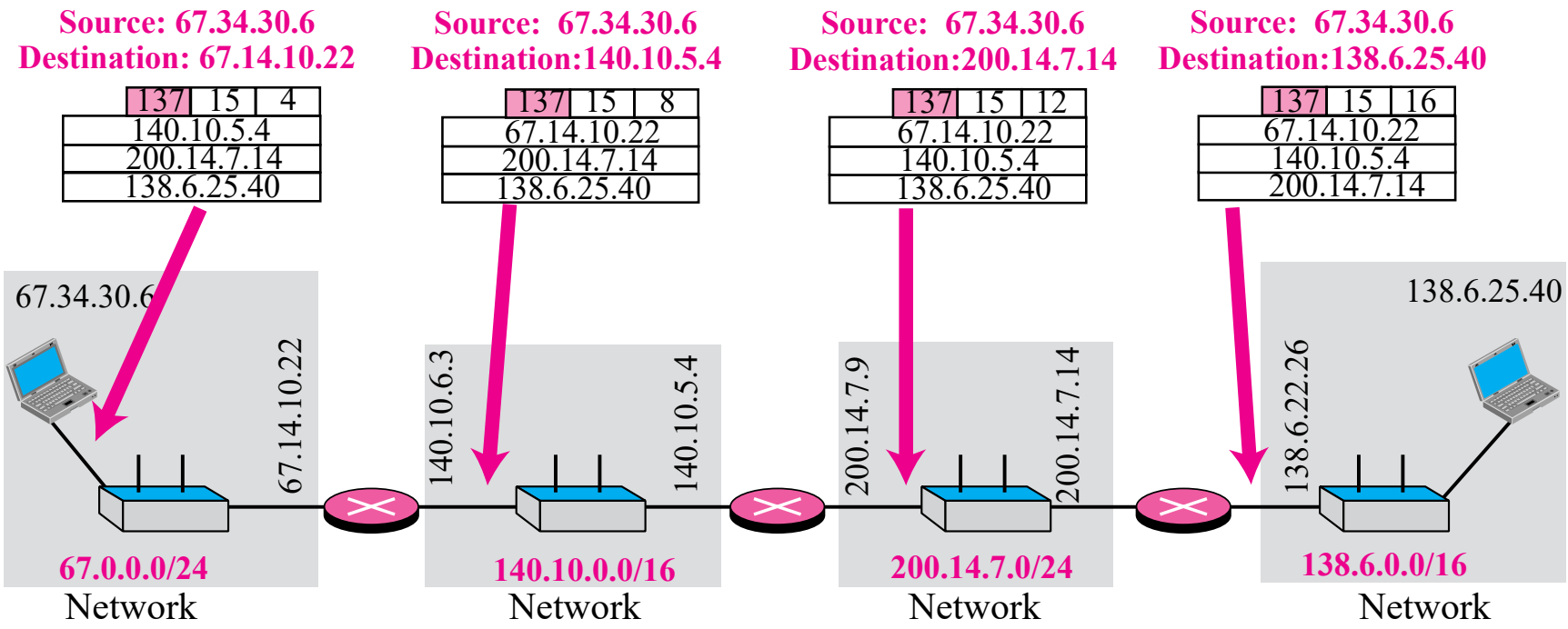
Record-route concept



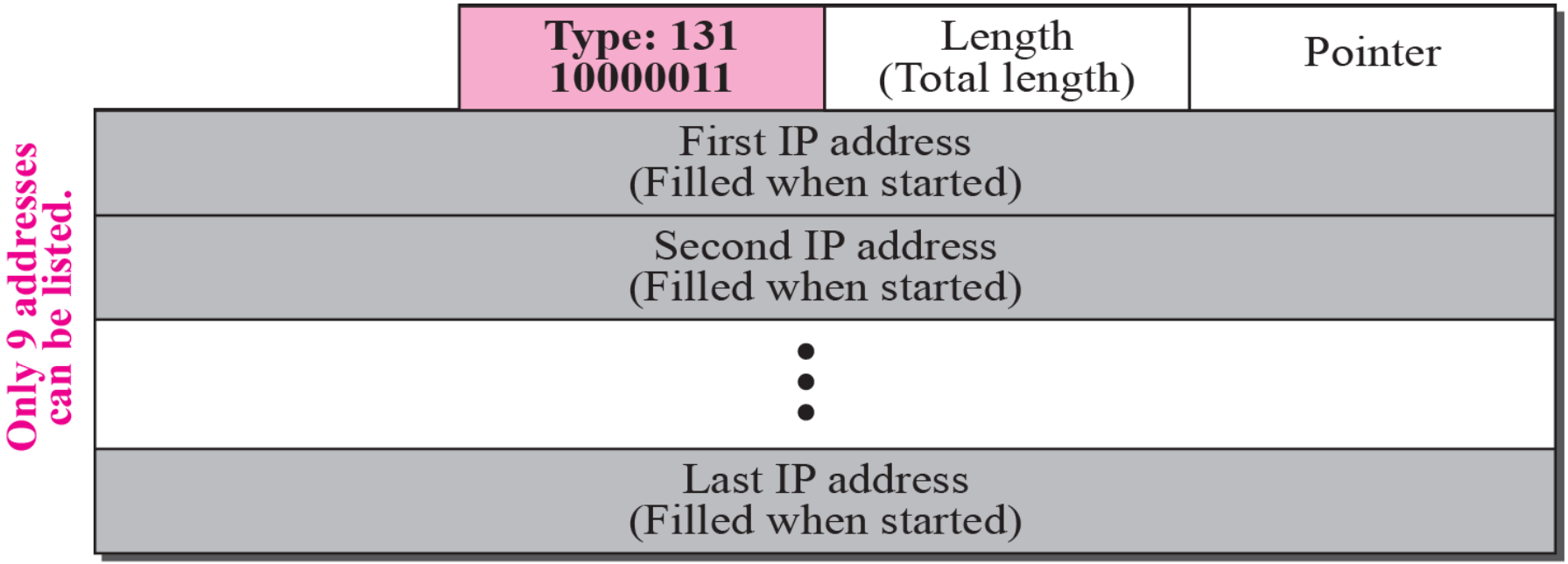
Strict-source-route option



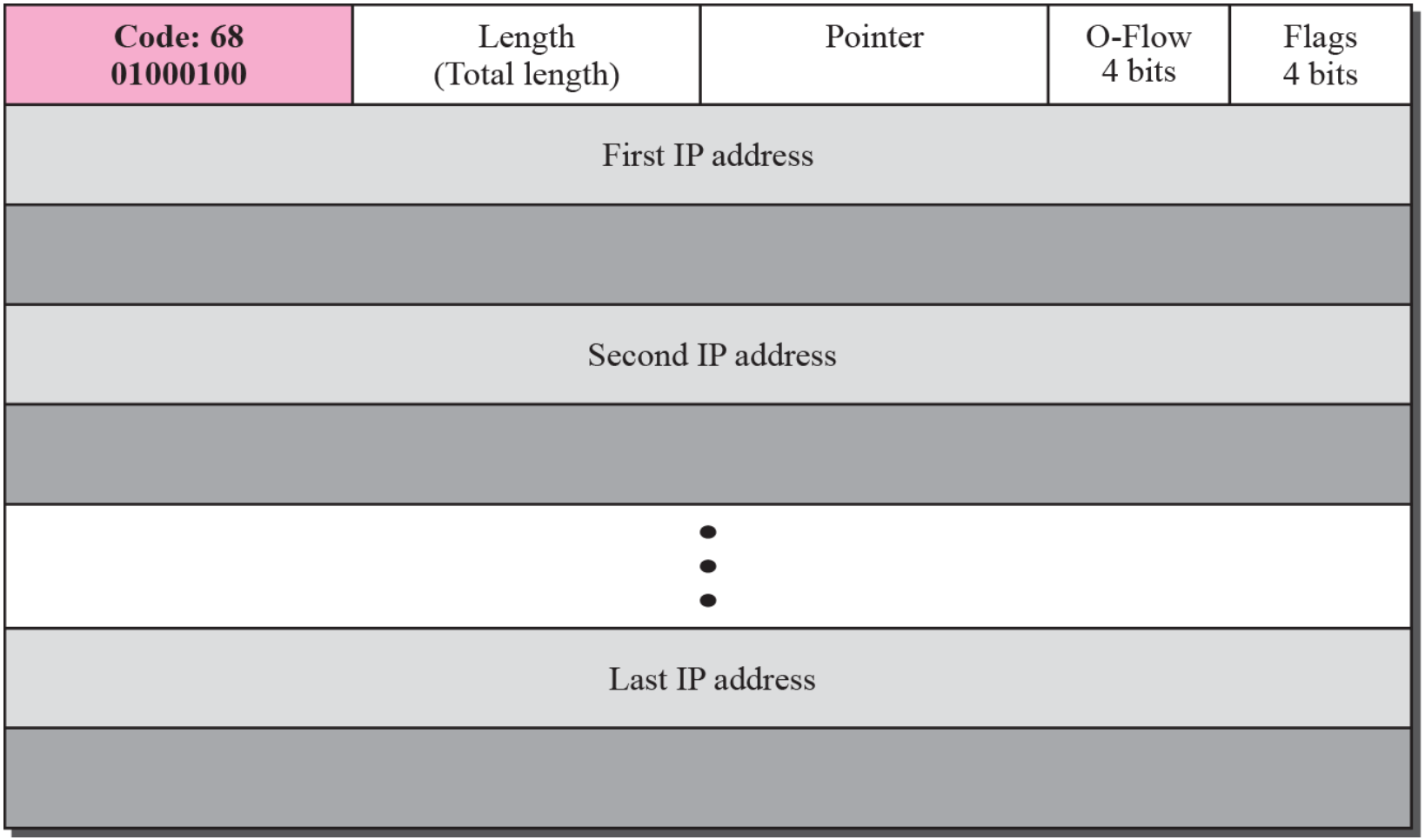
Strict-source-route option



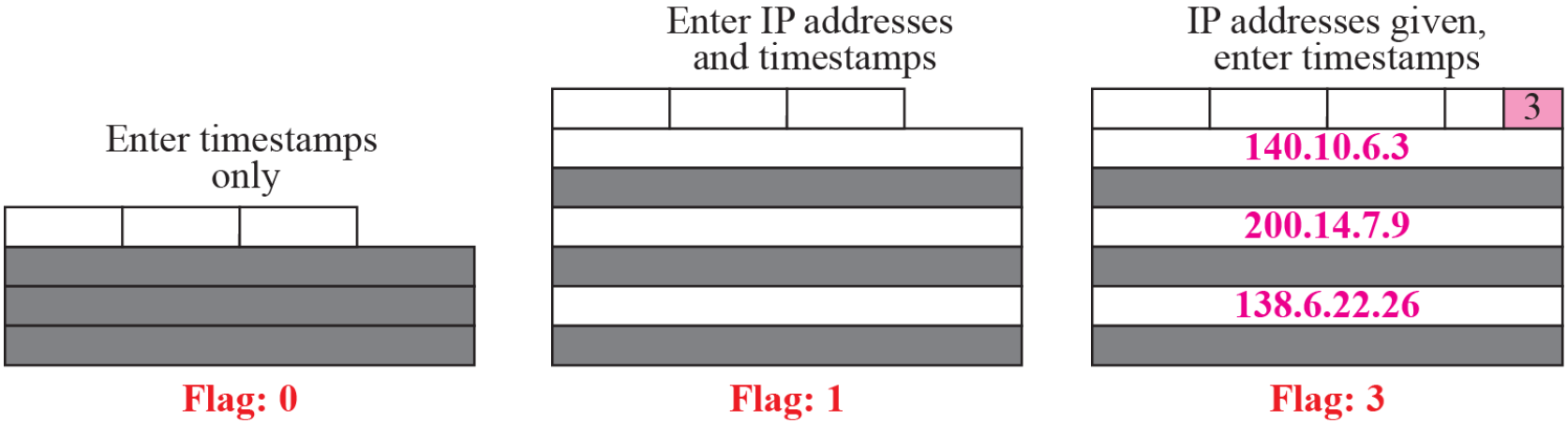
Loose-source-route option



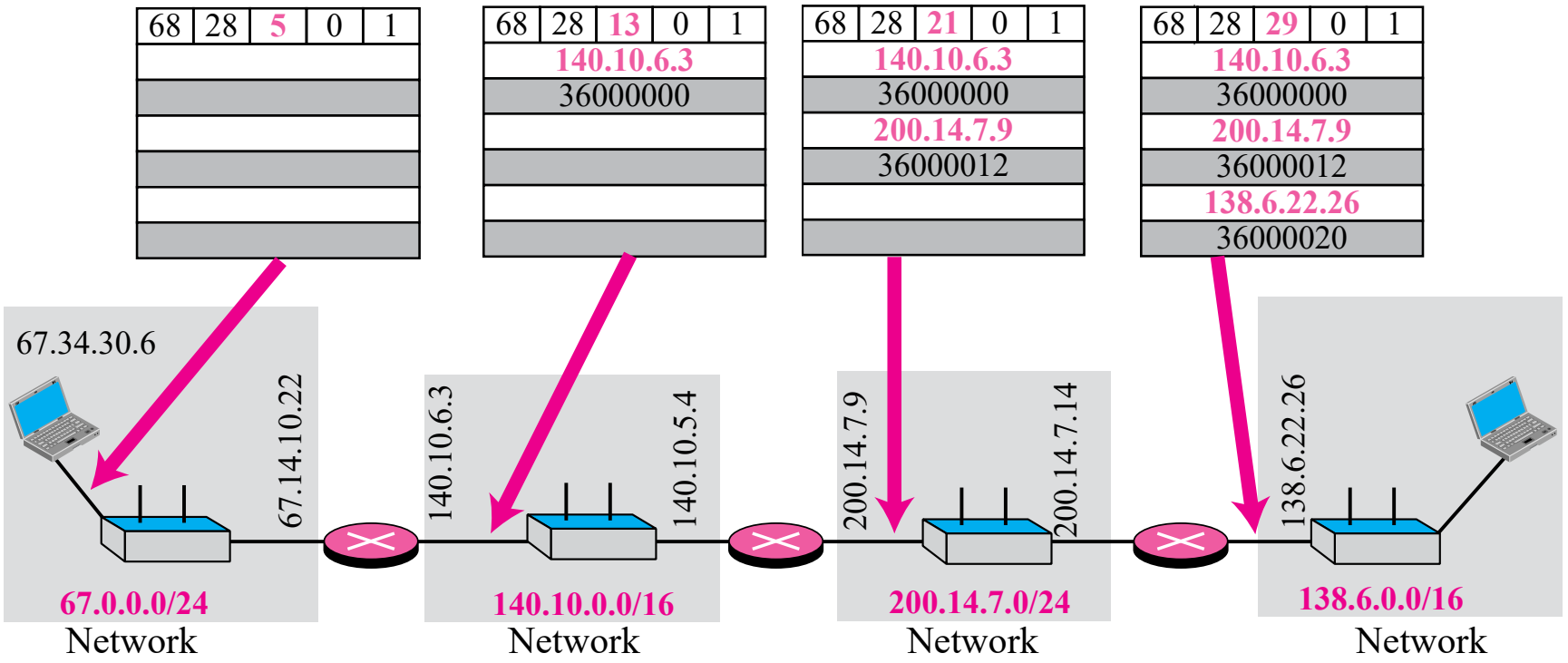
Time-stamp option



Use of flags in timestamp



Timestamp concept



Example

We can use the ping utility with the -R option to implement the record route option. The result shows the interfaces and IP addresses.

```
$ ping -R fhda.edu
PING fhda.edu (153.18.8.1) 56(124) bytes of data.
64 bytes from tiptoe.fhda.edu
(153.18.8.1): icmp_seq=0 ttl=62 time=2.70 ms
RR:  voyager.deanza.fhda.edu (153.18.17.11)
     Dcore_G0_3-69.fhda.edu (153.18.251.3)
     Dbackup_V13.fhda.edu (153.18.191.249)
     tiptoe.fhda.edu (153.18.8.1)
     Dbackup_V62.fhda.edu (153.18.251.34)
     Dcore_G0_1-6.fhda.edu (153.18.31.254)
     voyager.deanza.fhda.edu (153.18.17.11)
```

Example

The traceroute program can be used to implement loose source routing. The -g option allows us to define the routers to be visited, from the source to destination. The following shows how we can send a packet to the fhda.edu server with the requirement that the packet visit the router 153.18.251.4.

```
$ traceroute -g  153.18.251.4 fhda.edu.
traceroute to fhda.edu (153.18.8.1), 30 hops max, 46 byte packets
 1  Dcore_G0_1-6.fhda.edu (153.18.31.254)  0.976 ms  0.906 ms
    0.889 ms
 2  Dbackup_V69.fhda.edu (153.18.251.4)  2.168 ms  2.148 ms
    2.037 ms
```

Example

The traceroute program can also be used to implement strict source routing. The -G option forces the packet to visit the routers defined in the command line. The following shows how we can send a packet to the fhda.edu server and force the packet to visit only the router 153.18.251.4.

```
$ traceroute -G  153.18.251.4 fhda.edu.
traceroute to fhda.edu (153.18.8.1), 30 hops max, 46 byte packets
  1  Dbackup_V69.fhda.edu (153.18.251.4)  2.168 ms  2.148 ms
     2.037 ms
```

IP components

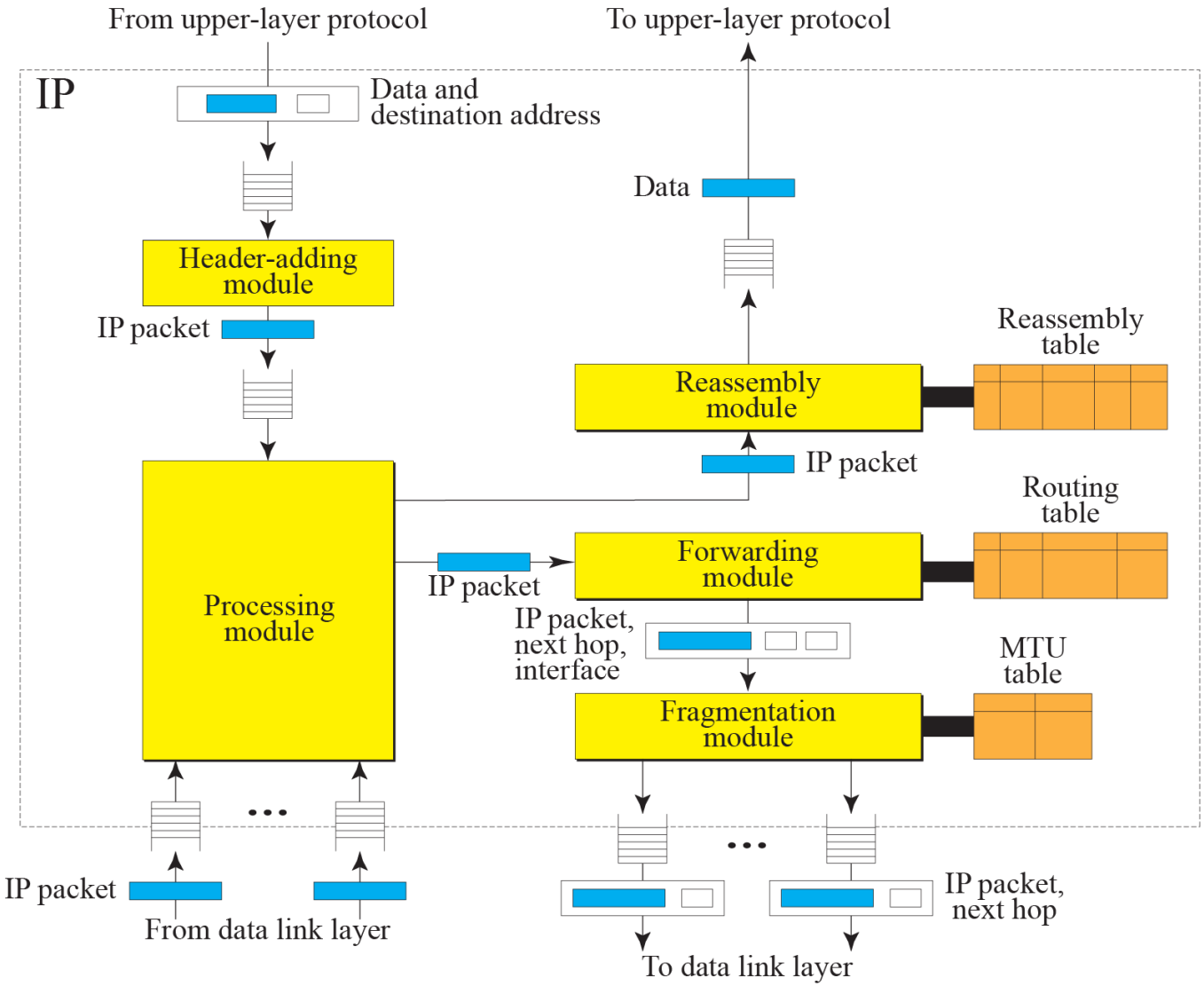


Table 7.3 *Adding module*

```
1  IP_Adding_Module (data, destination_address)
2  {
3       Encapsulate data in an IP datagram
4       Calculate checksum and insert it in the checksum field
5       Send data to the corresponding queue
6       Return
7  }
```

Table 7.4 *Processing module*

```
1   IP_Processing_Module (Datagram)
2   {
3         Remove one datagram from one of the input queues.
4         If (destination address matches a local address)
5         {
6             Send the datagram to the reassembly module.
7             Return.
8         }
9         If (machine is a router)
10        {
11            Decrement TTL.
12        }
13        If (TTL less than or equal to zero)
14        {
15            Discard the datagram.
16            Send an ICMP error message.
17            Return.
18        }
19        Send the datagram to the forwarding module.
20        Return.
21  }
```

Table 7.5 *Fragmentation module*

```
1   IP_Fragmentation_Module (datagram)
2   {
3        Extract the size of datagram
4        If (size > MTU of the corresponding network)
5        {
6             If (D bit is set)
7             {
8                  Discard datagram
9                  Send an ICMP error message
10                 return
11            }
12            Else
13            {
14                 Calculate maximum size
15                 Divide the segment into fragments
16                 Add header to each fragment
17                 Add required options to each fragment
```

Table 7.5 *Fragmentation module (continued)*

```
18                    Send fragment
19                    return
20               }
21          }
22          Else
23          {
24               Send the datagram
25          }
26          Return.
27    }
```

Reassembly table

St.: State
S. A.: Source address
D. I.: Datagram ID
T. O.: Time-out
F.: Fragments

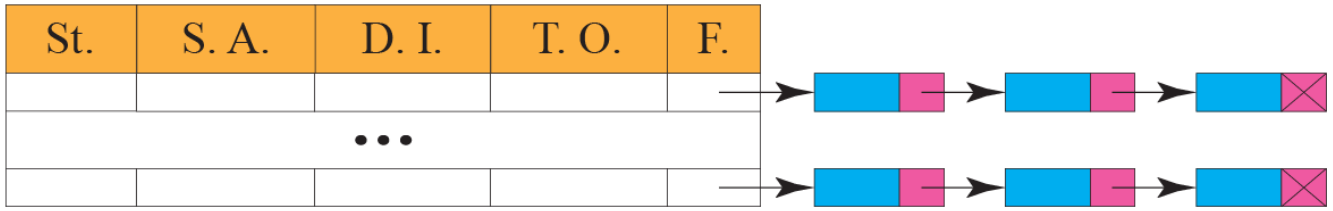
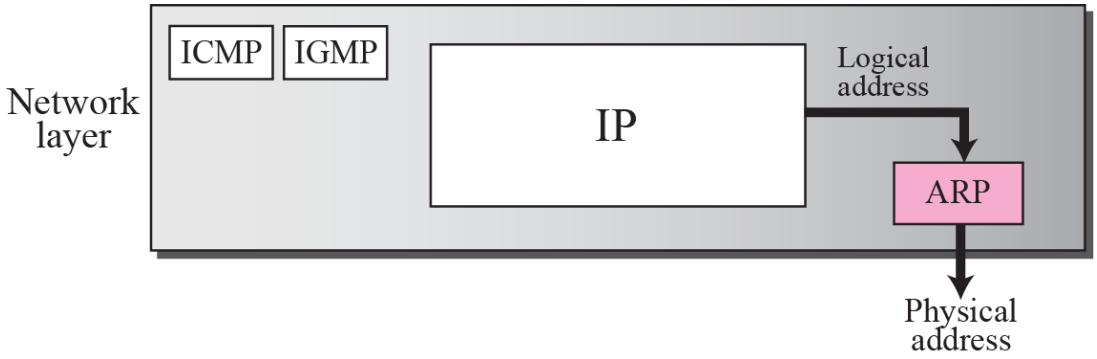


Table 7.6 *Reassembly module*

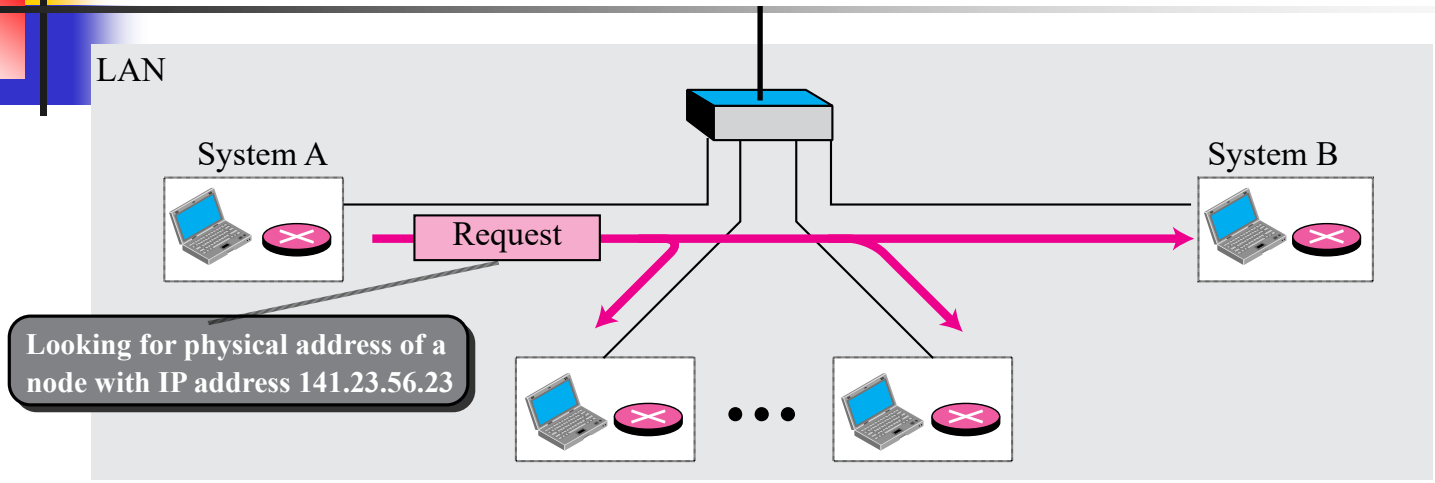
```
1   IP_Reassembly_Module (datagram)
2   {
3        If (offset value = 0 AND M = 0)
4        {
5             Send datagram to the appropriate queue
6             Return
7        }
8        Search the reassembly table for the entry
9        If (entry not found)
10       {
11            Create a new entry
12       }
13       Insert datagram into the linked list
14       If (all fragments have arrived)
15       {
16            Reassemble the fragment
17            Deliver the fragment to upper-layer protocol
18            return
19       }
20       Else
21       {
22            If (time-out expired)
23            {
24                 Discard all fragments
25                 Send an ICMP error message
26            }
27       }
28       Return.
29  }
```

ARP: ADDRESS MAPPING

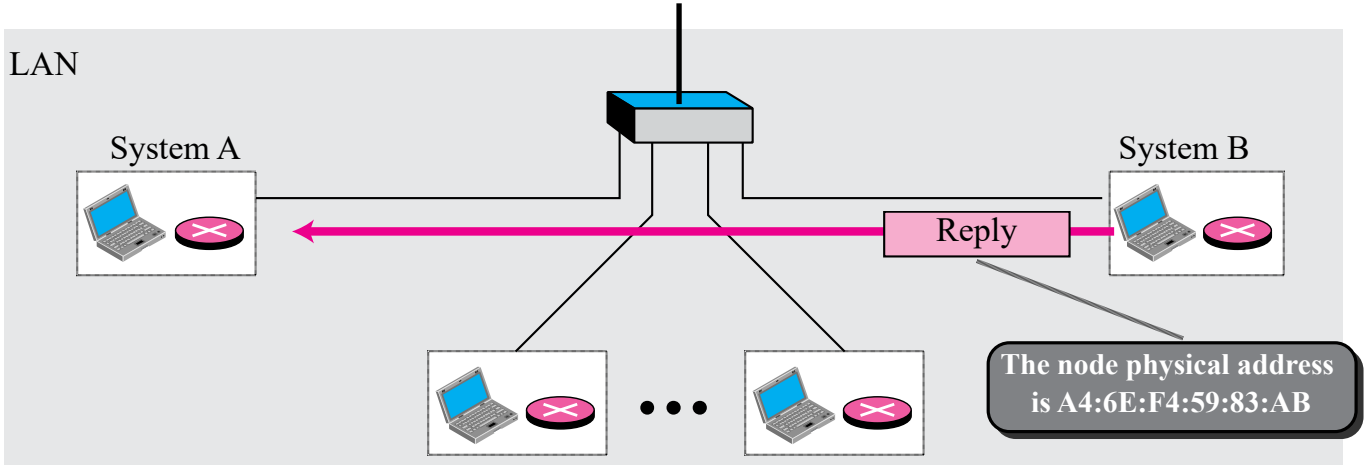
- IP datagram must be encapsulated in a frame to be able to pass through the physical network.
- sender needs the physical address of the receiver.
- ARP accepts a logical address from the IP protocol, maps the address to the corresponding physical address and pass it to the data link layer.



ARP operation



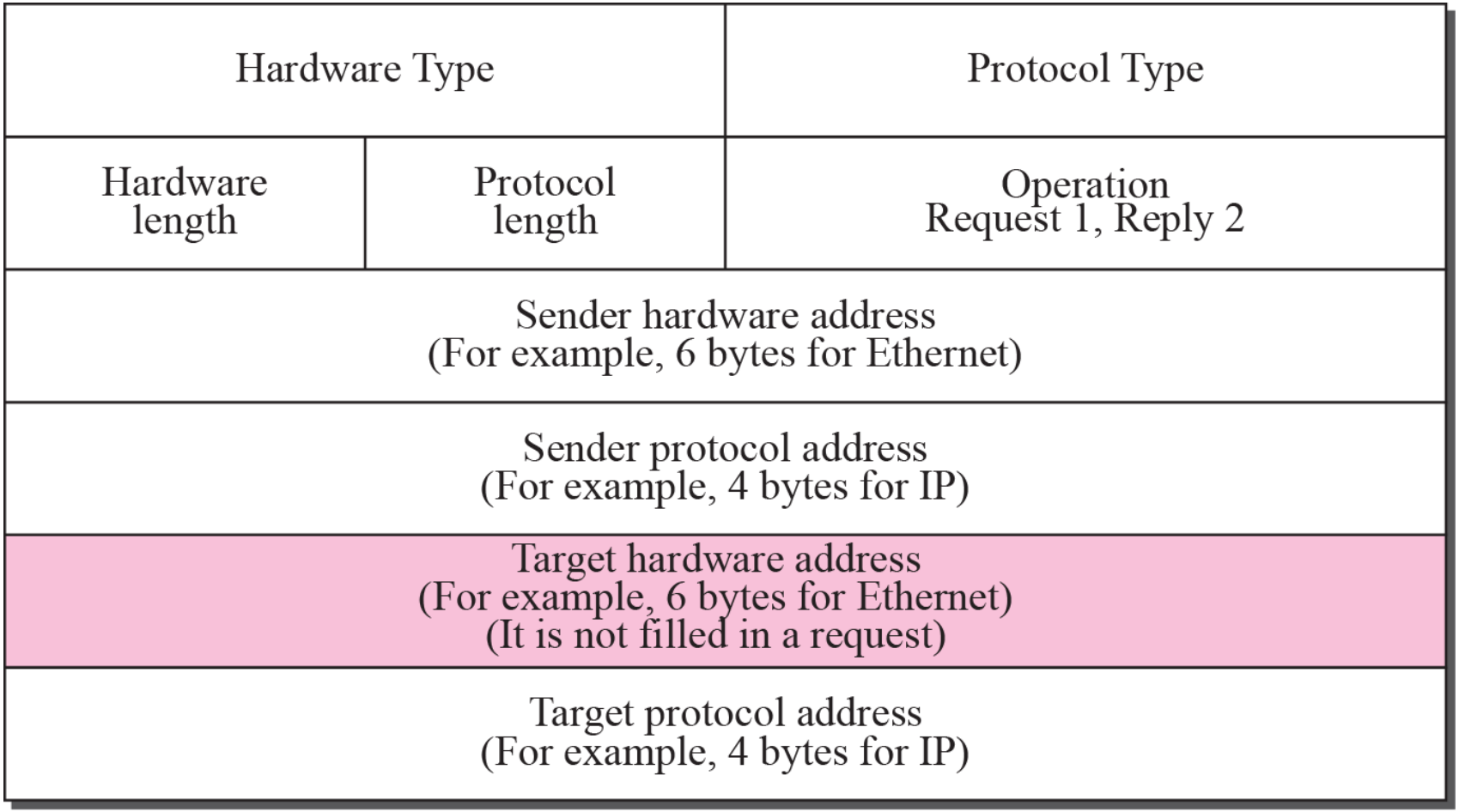
a. ARP request is multicast



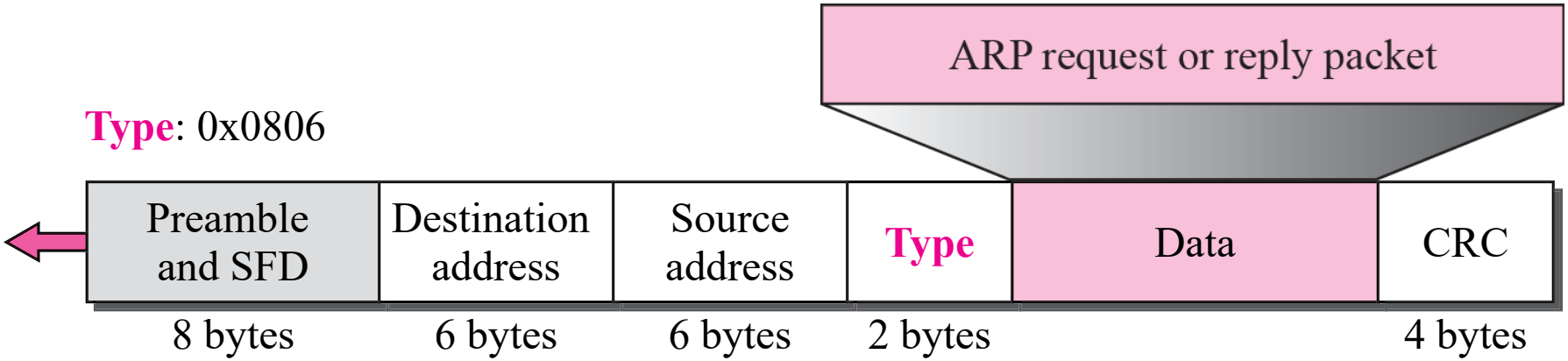
b. ARP reply is unicast

An ARP request is broadcast; an ARP reply is unicast.

ARP packet

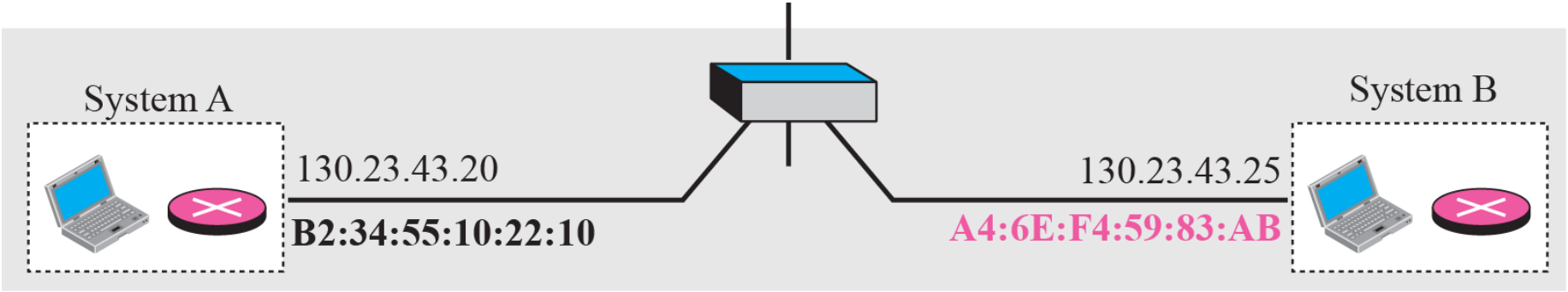

Hardware Type
Protocol Type
Hardware length
Protocol length
Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)
Sender protocol address (For example, 4 bytes for IP)
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)
Target protocol address (For example, 4 bytes for IP)

Encapsulation of ARP packet

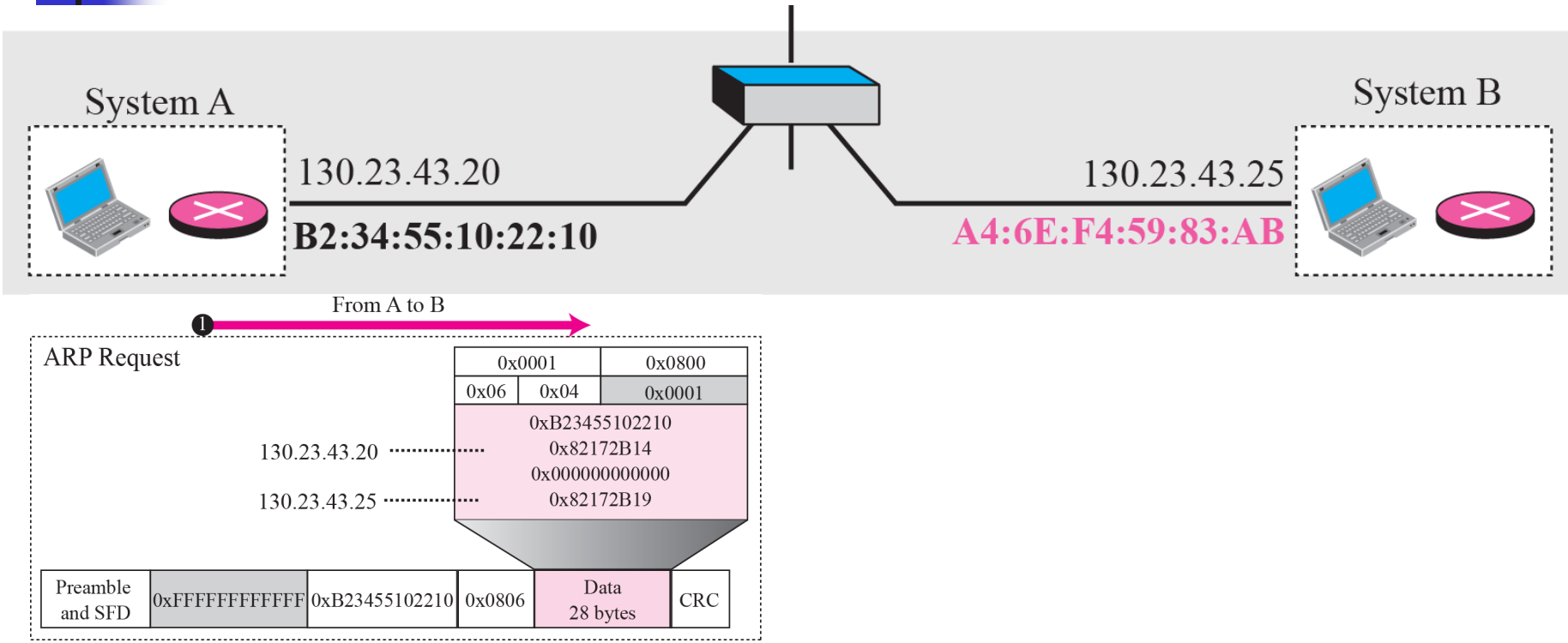


Example

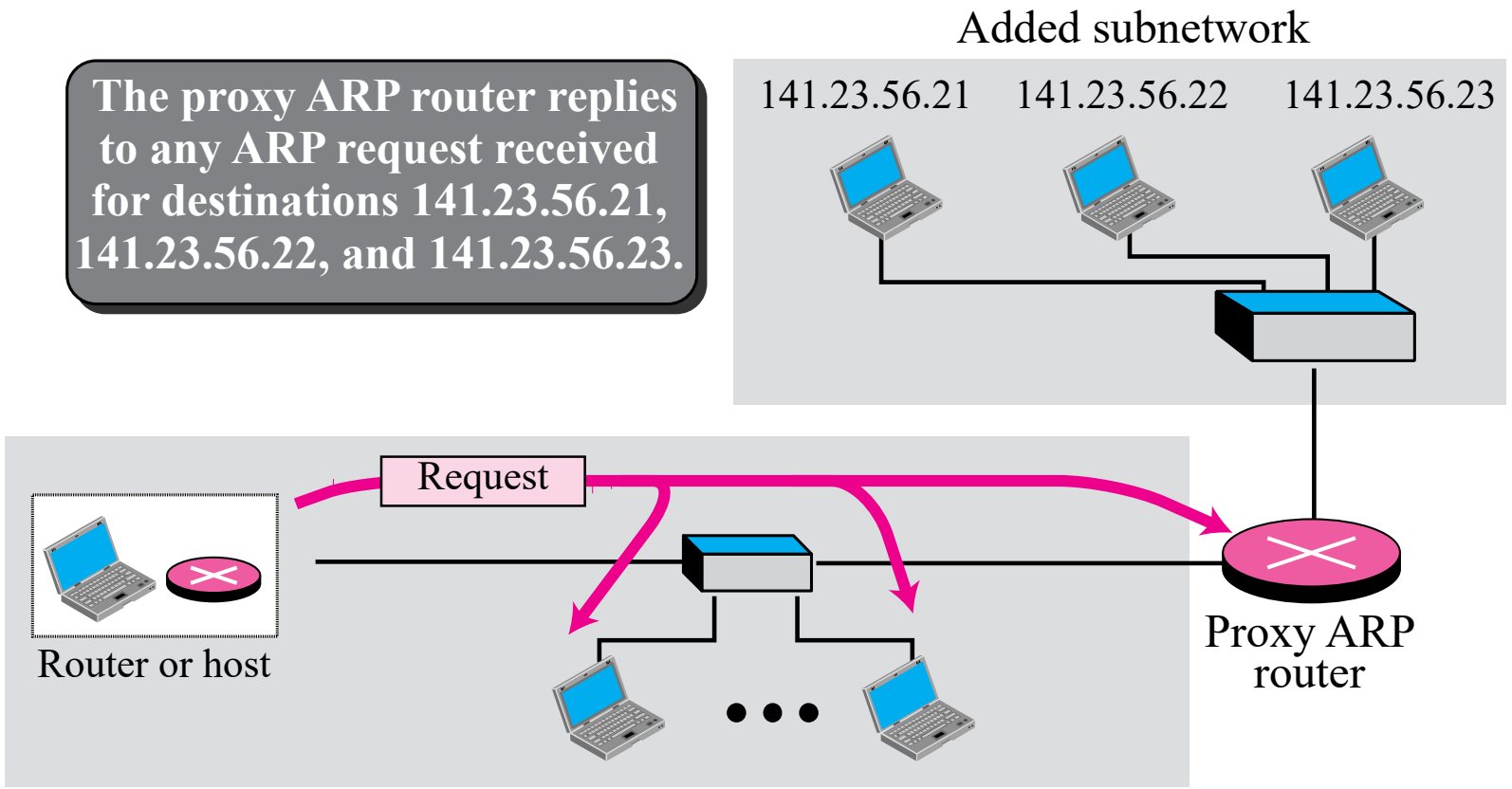
A host with IP address 130.23.43.20 and physical address B2:34:55:10:22:10 has a packet to send to another host with IP address 130.23.43.25 and physical address A4:6E:F4:59:83:AB. The two hosts are on the same Ethernet network. Show the ARP request and reply packets encapsulated in Ethernet frames.



Example


System A
System B
130.23.43.20
B2:34:55:10:22:10
130.23.43.25
A4:6E:F4:59:83:AB
From A to B
1
ARP Request
0x0001
0x0800
0x06
0x04
0x0001
0xB23455102210
0x82172B14
0x000000000000
0x82172B19
130.23.43.20
130.23.43.25
Preamble and SFD
0xFFFFFFFFFFFF
0xB23455102210
0x0806
Data 28 bytes
CRC
From B to A
2
ARP Reply
0x0001
0x0800
0x06
0x04
0x0002
0xA46EF45983AB
0x82172B19
0xB23455102210
0x82172B14
130.23.43.25
130.23.43.20
Preamble and SFD
0xB23455102210
0xA46EF45983AB
0x0806
Data
CRC

The proxy ARP router replies to any ARP request received for destinations 141.23.56.21, 141.23.56.22, and 141.23.56.23.



References

TCP/IP Protocol Suite (4th Edition),

by Behrouz A. Forouzan – Used the text's slide deck

End of session